**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**
**Ордена трудового Красного Знамени федеральное государственное бюджетное**
**образовательное учреждение высшего образования**
**«Московский технический университет связи и информатики»**

Кафедра Математическая кибернетика и информационные технологии

# Лабораторная работа №3

## Разработка веб-приложения на Spring Boot.

## Введение в IoC/DI

Выполнил: студент группы ___БВТ2402___

Голиков Михаил Вячеславович

Руководитель: _Мосева Марина Сергеевна_

Москва, 2077

## Цель работы

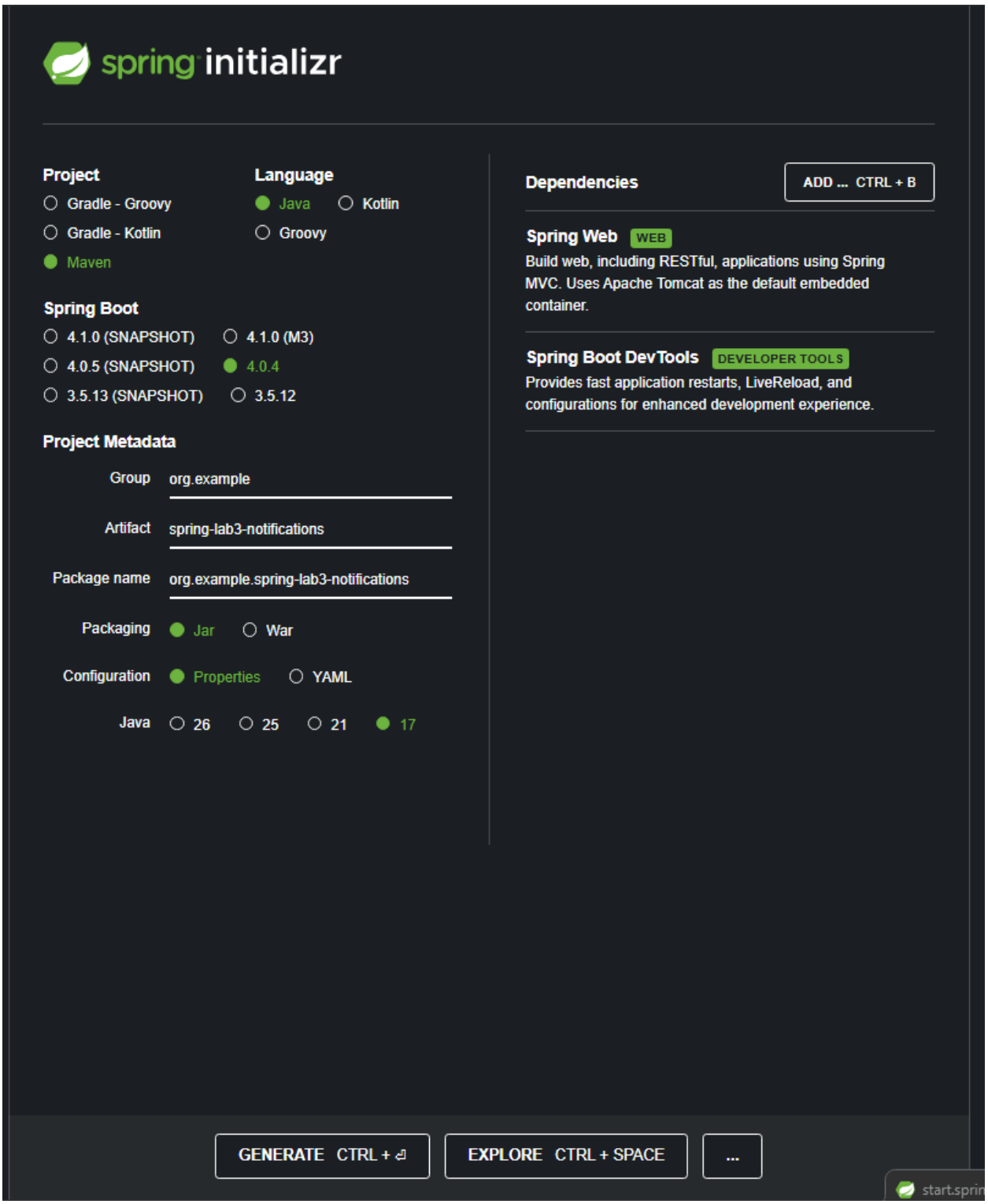
Научиться создавать Spring Boot приложение, освоить принципы инверсии управления (IoC) и внедрения зависимостей (DI), изучить три способа конфигурации Springкомпонентов, а также научиться разрешать конфликты при наличии нескольких бинов одного типа.

## Индивидуальное задание

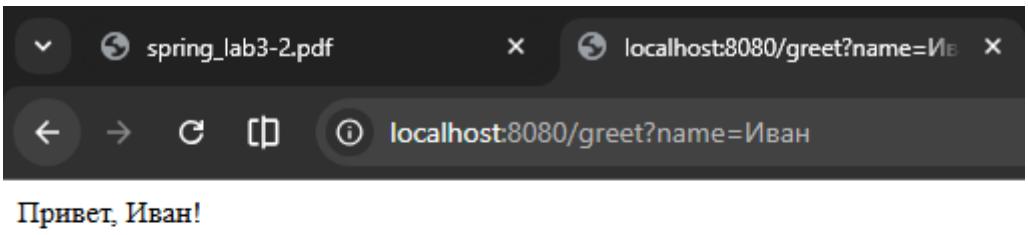
На основе выполненной лабораторной работы разработайте систему уведомлений, которая поддерживает несколько каналов доставки и позволяет динамически выбирать нужный канал на основе входных параметров. Создайте интерфейс NotificationChannel с методом send, реализуйте три канала: EmailChannel, SmsChannel и PushChannel. Добавьте сервис NotificationRouter, который принимает Map<String, NotificationChannel> и на основе переданного параметра channelType выбирает соответствующий канал для отправки. Настройте приложение таким образом, чтобы все каналы автоматически обнаруживались Spring через механизм компонент-сканирования, а выбор канала происходил без использования условных операторов if-else. Дополнительно реализуйте возможность отправки уведомления одновременно во все каналы, если в запросе передан специальный параметр. Убедитесь, что при добавлении нового канала доставки не требуется изменять существующий код сервиса маршрутизации, что демонстрирует соблюдение принципа открытости-закрытости.

## Основная часть

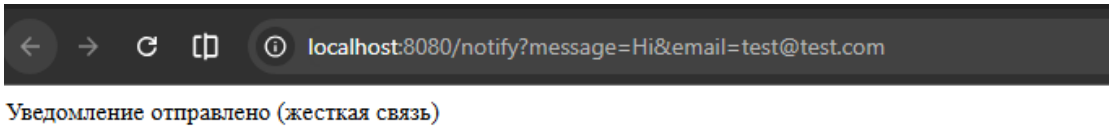
В ходе выполнения лабораторной работы были выполнены все пункты для изучения Spring Boot.




Элемент 1 — Создание Spring репозитория




Элемент 2 — Первый запуск web приложения




Элемент 3 — Создание жесткой связи

```
package org.example.service;

public class PushService implements MessageService {  no usages  new *
    @Override  1 usage  new *
    public void sendMessage (String message, String recipient) {
        System.out.println("Push to " + recipient + " : " + message);
    }
}
```

Элемент 4 — Добавление работы PushService

```java
package org.example.controller;
import org.example.service.MessageService;
import org.example.service.NotificationManager;
import org.example.service.PushService;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@RestController  no usages
public class NotificationController {

    @GetMapping("/notify")  no usages
    public String notify(@RequestParam String message, @RequestParam String recipient) {
        MessageService pushService = new PushService();
        NotificationManager manager = new NotificationManager(pushService);
        manager.notify(message, recipient);
        return "Уведомление отправлено (Жёсткая связь)";
    }
}
```

Элемент 5 — Добавление жесткой связи

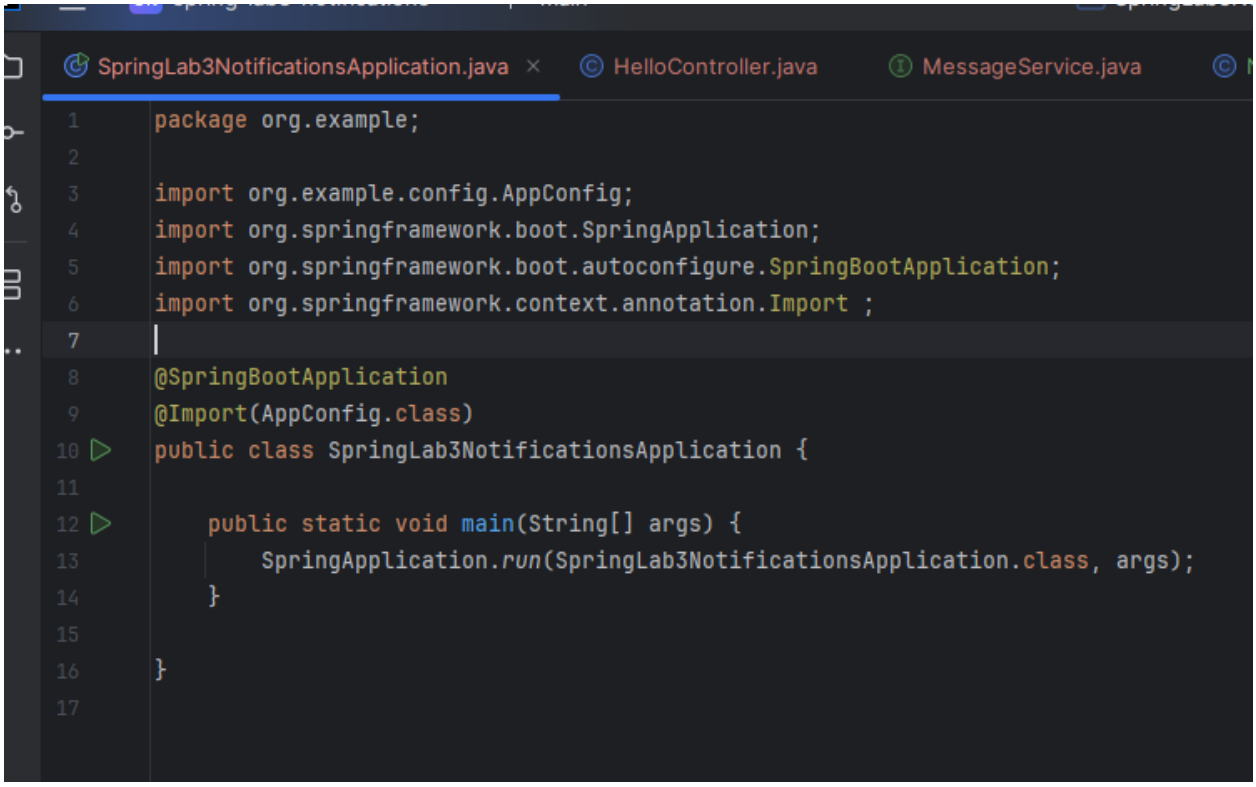
```java
package org.example.service;

public class NotificationManager {  3 usages  new *

    private final MessageService messageService;  2 usages
    public NotificationManager(MessageService messageService) {  1 usage  new *
        this.messageService = messageService;
    }

    public void notify(String message, String recipient) {  1 usage  new *
        messageService.sendMessage(message, recipient);
    }

}
```

Элемент 6 — Создание NotifyManager

После изменения `NotificationManager` код всё равно остаётся неудобным. В контроллере приходится вручную создавать сервисы, что делает

его громоздким. При смене реализации нужно править все места вызова конструкторов. Нет централизованного управления объектами — каждый создаёт свои экземпляры, что мешает использовать синглтоны. Тестирование усложняется, так как зависимости приходится настраивать вручную. В больших проектах это приводит к путанице в коде и проблемам с поддержкой.

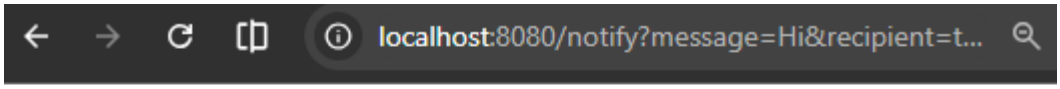
Элемент 7 — Работа с Java-config




Элемент 8 — Импорт конфига

```java
package org.example.config;

import org.example.service.EmailService;
import org.example.service.NotificationManager;
import org.example.service.PushService;
import org.example.service.SmsService;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class AppConfig {

    @Bean
    public EmailService emailService() {
        return new EmailService();
    }

    @Bean
    public SmsService smsService() {
        return new SmsService();
    }

    @Bean
    public PushService pushService() {
        return new PushService();
    }

    @Bean
    public NotificationManager notificationManager() {
        // ЯВНО указываем, какой бин внедряем
        return new NotificationManager(emailService());
    }
}
```

Элемент 9 — Написание конфига




Уведомление отправлено через аннотации

Элемент 10 — Работа через аннотации

```java
package org.example.config;

import org.example.service.EmailService;
import org.example.service.NotificationManager;
import org.example.service.PushService;
import org.example.service.SmsService;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.ComponentScan;

@Configuration  1 usage  new *
@ComponentScan ("org.example")
@ComponentScan ("org.example.controller")
public class AppConfig {

//    @Bean
//    public EmailService emailService() {
//        return new EmailService();
//    }
//
//    @Bean
//    public SmsService smsService() {
//        return new SmsService();
//    }
//
//    @Bean
//    public PushService pushService() {
//        return new PushService();
//    }
//
//    @Bean
//    public NotificationManager notificationManager() {
//        // ЯВНО указываем, какой бин внедряем
//        return new NotificationManager(emailService());
//    }
}
```

Элемент 11 — Исправление конфига для аннотаций

Ошибка возникает из-за того, что Spring находит два бина одного типа — EmailService и SmsService, оба реализуют интерфейс MessageService. Когда Spring пытается внедрить зависимость в конструктор NotificationManager, он не может определить, какой именно бин использовать, и выбрасывает исключение NoUniqueBeanDefinitionException.

Если временно удалить аннотацию @Service с одного из классов-сервисов, например с SmsService, то в контексте останется только один бин типа MessageService — EmailService. Приложение запустится без ошибок, и уведомления будут отправляться через оставшийся сервис. Это доказывает, что проблема именно в наличии двух кандидатов одного типа.

Если изменить аннотацию на @ComponentScan("org.example.controller"), то Spring будет сканировать только пакет controller и его подпакеты. Бины из пакета service не будут обнаружены, и приложение упадёт с ошибкой, потому что NotificationManager не будет найден в контексте. Это показывает, что @ComponentScan указывает Spring, где искать классы с аннотациями @Component, @Service, @Repository и @Controller.

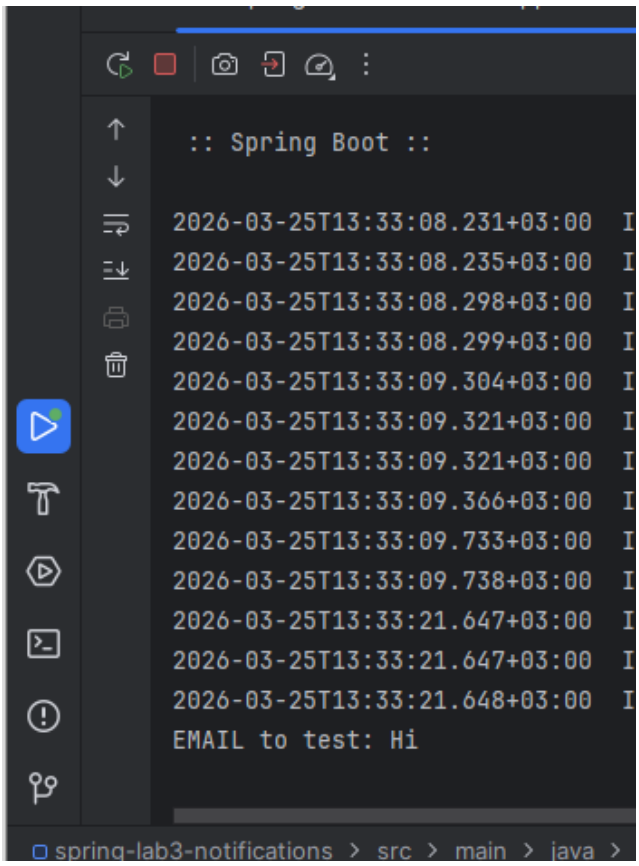
```
package org.example.service;

public interface AdvancedMessageService extends MessageService {
    String getServiceType();  no usages  1 implementation  new *
}
```

Элемент 12 — Добавление продвинутого интерфейса

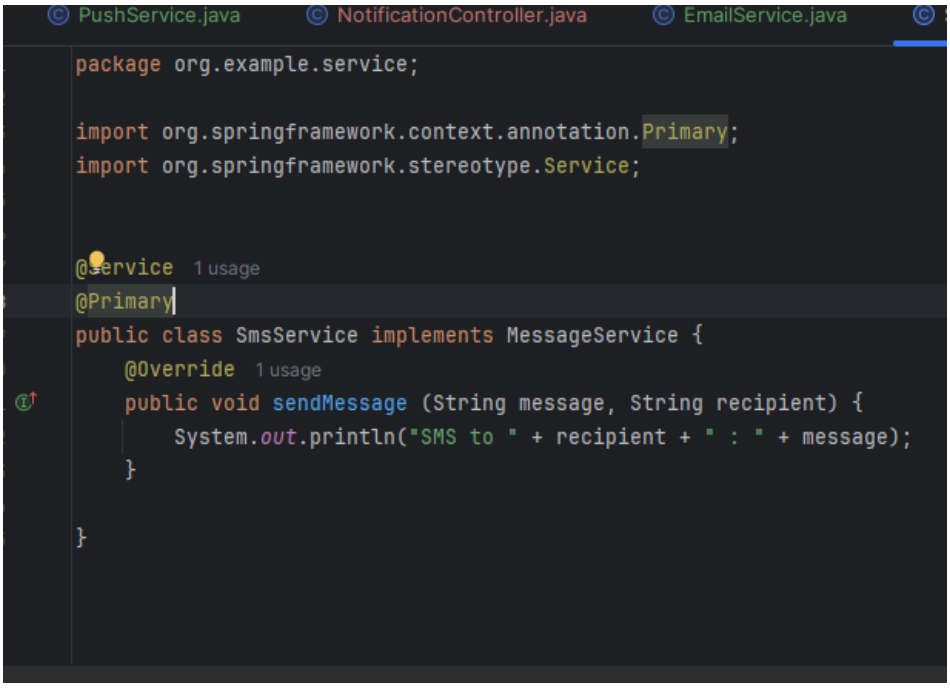
```java
package org.example.service;

import org.springframework.stereotype.Service;

@Service  1 usage  new *
public class EmailService implements AdvancedMessageService {
    @Override  1 usage  new *
    public void sendMessage(String message, String recipient) {
        System.out.println("EMAIL to " + recipient + ": " + message);
    }

    @Override  no usages  new *
    public String getServiceType() {
        return "Email Service";
    }
}
```

Элемент 13 — Исправление EmailService для нового интерфейса

Это не повлияет на конфликт напрямую, так как Spring всё равно будет видеть два бина, реализующих MessageService. Однако если изменить тип поля в NotificationManager на AdvancedMessageService, то Spring сможет внедрить только тот бин, который реализует этот интерфейс, и конфликт разрешится автоматически.




Элемент 14 — С продвинутым интерфейсом Spring Boot сам определил единственный подходящий бин




Элемент 15 — Добавление ключа Primary

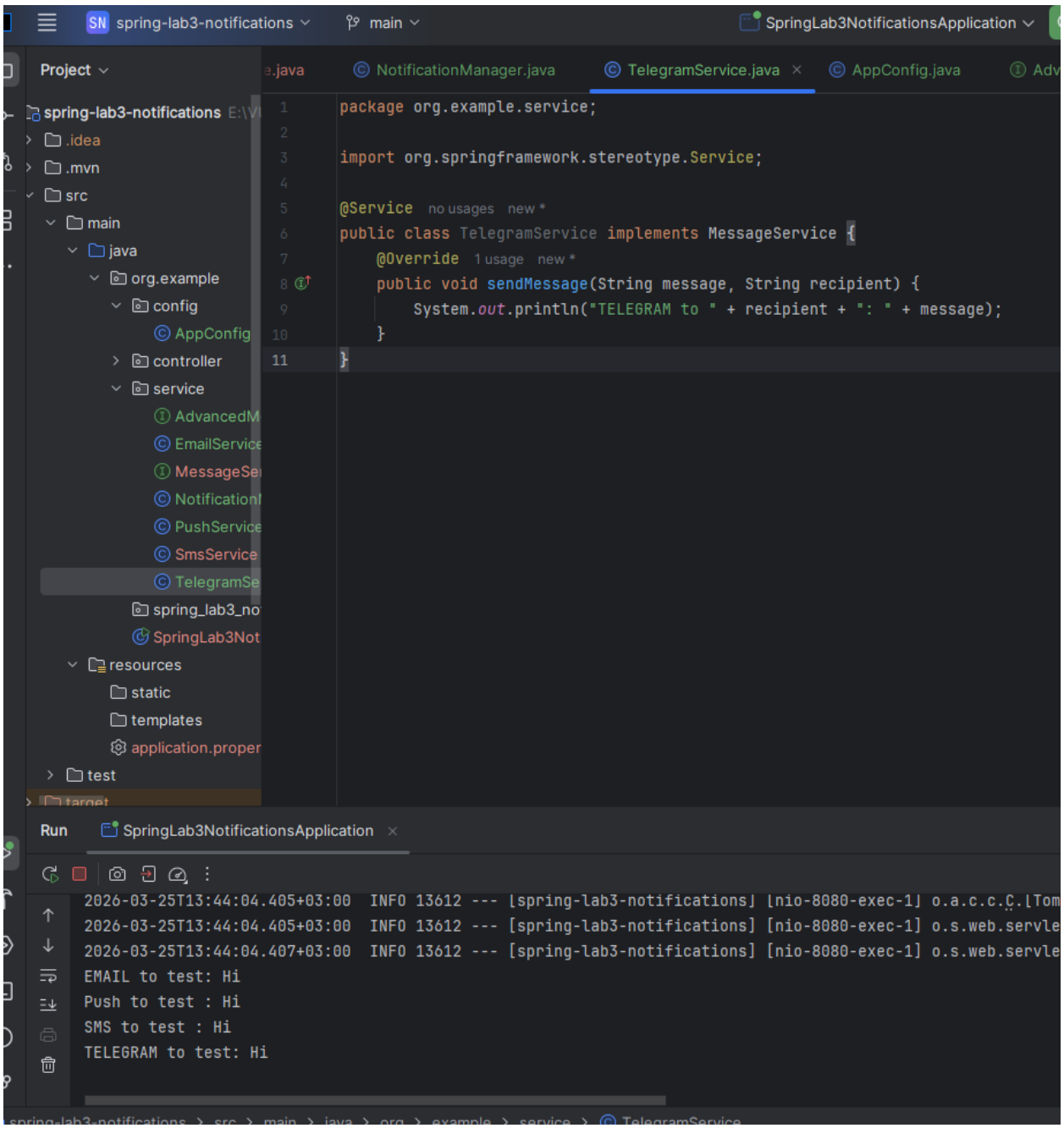
```java
package org.example.service;

import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.stereotype.Service;
import org.springframework.beans.factory.annotation.Autowired;

@Service  4 usages  new *
public class NotificationManager {
//    private final AdvancedMessageService messageService;
//
//    @Autowired
//    public NotificationManager(AdvancedMessageService messageService) {
//        this.messageService = messageService;
//    }

    private final MessageService messageService;  2 usages

    @Autowired  no usages  new *
    public NotificationManager(@Qualifier("smsService") MessageService messageService) {
        this.messageService = messageService;
    }

    public void notify(String message, String recipient) {  1 usage  new *
        messageService.sendMessage(message, recipient);
    }

}
```

Элемент 16 — добавление Qualifier

```
@Service  4 usages  new *
public class NotificationManager {
//    private final AdvancedMessageService messageService;
//
//    @Autowired
//    public NotificationManager(AdvancedMessageService messageService) {
//        this.messageService = messageService;
//    }

    private final List<MessageService> messageService;  2 usages

//    @Autowired
//    public NotificationManager(@Qualifier("smsService") MessageService messageService) {
//        this.messageService = messageService;
//    }

    @Autowired  no usages  new *
    public NotificationManager(List<MessageService> messageService) {
        this.messageService = messageService;
    }

    public void notify(String message, String recipient) {  1 usage  new *
//        messageService.sendMessage(message, recipient);
        messageService.forEach( MessageService service -> service.sendMessage(message, recipient));
    }

}
```

```
2026-03-25T13:40:34.082+03:00  INFO 18980 --- [spring-lab3-notifications] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet
EMAIL to test: Hi
Push to test : Hi
SMS to test : Hi
```

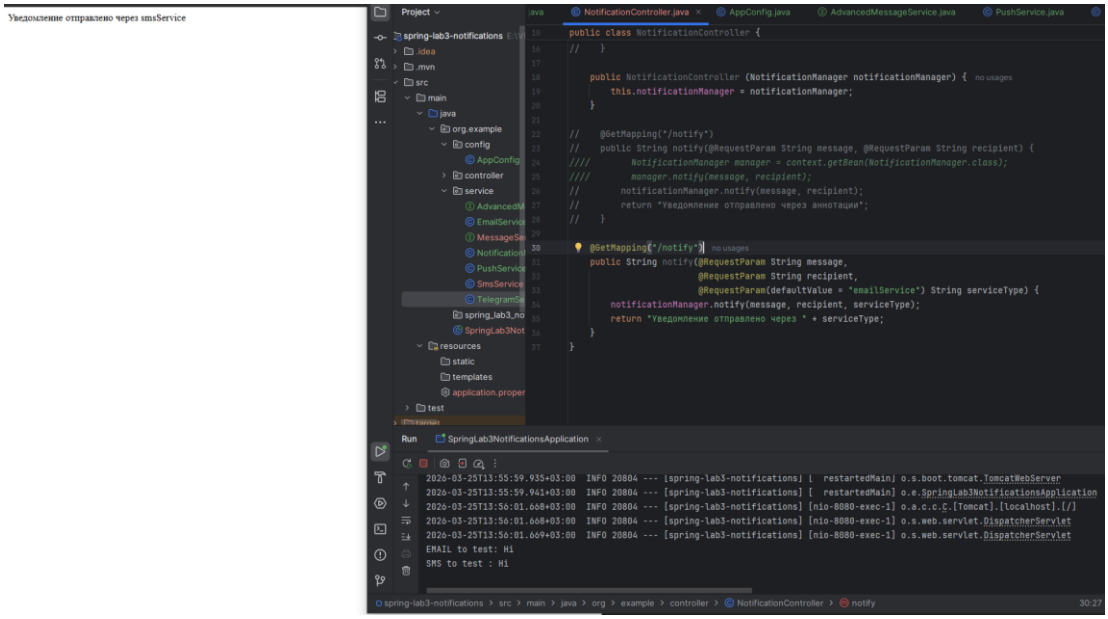
Элемент 17 — Работа со всеми реализациями интерфейса через List




Элемент 18 — Добавление телеграмм реализации

Если одновременно использовать @Primary на одном сервисе и внедрение List<MessageService> в другом месте, это работает следующим образом: @Primary влияет только на места, где ожидается одиночный бин типа MessageService и Spring должен сделать выбор по умолчанию. Внедрение коллекции List<MessageService> при этом не учитывает @Primary — в список попадают все бины независимо от того, какой из них помечен как основной.

Таким образом, эти два механизма работают независимо и не конфликтуют друг с другом.




Элемент 19 — Пример работы сервиса по выбору

```
    public void notify(String message, String recipient, String serviceType) {
        MessageService service = messageServices.get(serviceType);
        if (service != null) {
            service.sendMessage(message, recipient);
        } else {
            System.out.println("Сервис типа " + serviceType + " не найден");
        }
    }
}
```

Элемент 20 — Реализация работы сервиса по выбору

## Заключение

В ходе выполнения лабораторной работы были успешно освоены ключевые принципы Spring Framework — инверсия управления и внедрение зависимостей. На примере системы уведомлений были наглядно продемонстрированы недостатки подхода с жёсткими связями, когда классы самостоятельно создают свои зависимости, что приводит к сложностям при тестировании, масштабировании и изменении логики приложения. Переход к использованию Spring IoC-контейнера позволил централизованно управлять созданием и связыванием объектов, делая код более гибким и поддерживаемым. Были изучены три способа конфигурации Spring-компонентов: явная конфигурация через @Configuration и @Bean, автоматическое обнаружение через @ComponentScan с аннотациями @Service, а также разрешение конфликтов при наличии нескольких бинов одного типа с помощью @Primary, @Qualifier и внедрения коллекций. Наиболее гибким подходом оказалось внедрение коллекции или карты всех доступных реализаций, что позволяет динамически выбирать нужный сервис во время выполнения. Внедрение зависимостей через конструктор показало свои преимущества перед полевым внедрением, так как делает зависимости явными, неизменяемыми и упрощает тестирование. Полученные навыки являются фундаментальными для разработки промышленных Spring-приложений с соблюдением принципов слабой связанности и высокой расширяемости.

**Github:** https://github.com/Prototype721/Java